



Progettazione di applicazioni web full stack

Sviluppo backend con Node e NestJS



Prof. Devis Bianchini
Dip. di Università degli Studi di Brescia



Argomento di questa lezione...

Cosa faremo

- Caratteristiche ed elementi principali di **NestJS**
- Come utilizzare **NestJS** dentro **Nx**
- Configurare un progetto **NestJS** in **Nx**
- Esporre le funzionalità di backend come REST API

Output

- Primo progetto backend in **node** utilizzando **NestJS** e **Nx**
- Pubblicazione delle REST API sviluppate in **NestJS** tramite **Swagger**

Cos'è NestJS?



- **NestJS** è un framework basato su **node** e TS/JS
 - *Opinionated*
 - Ampia documentazione
 - Supportato da una comunità molto attiva
- Combina aspetti di
 - *Object-Oriented Programming*
 - *Functional (Reactive) Programming*

Altri framework spesso accostati a **NestJS**:

- **NextJS** - è un framework basato su **React**
- **Express** - è un framework per node *unopinionated* (su cui **NestJS** è costruito)

I componenti di NestJS



- **Controller** – espone rotte (HTTP) e riceve le richieste, delegando la logica ai provider e restituendo la risposta
- **Provider** – incapsula la logica di business e le dipendenze riutilizzabili, può essere *iniettato* come una dipendenza in altre classi (*servizi*, *repositories*, *factories*, *helpers* possono essere implementati come provider)
- **Module** – organizza la app o la libreria in unità funzionali, dichiarando controller e provider e definendo cosa viene *importato* e cosa viene *esportato*



- Non si tratta di due strumenti incompatibili, anzi...
 - **Nx** è un toolkit per gestire *monorepo* e *tooling di sviluppo* (workspace, build/test, dependency graph)
 - **Nx** può ospitare un progetto **NestJS** dentro un monorepo, aiuta a
 - Creare app **NestJS** e librerie condivise
 - Organizzare codice condiviso
 - Imporre confini tra moduli/librerie
- Utile utilizzare insieme **NestJS** e **Nx** quando il progetto è articolato e *multi-app* (backend/shared_libs **NestJS/TypeScript**, frontend **React**)

Creare il primo progetto NestJS



- Installare il plugin **NestJS** (vd lezione introduttiva a **Nx**)
- Creare una nuova app con **NestJS**

```
npx nx add @nx/nest  
  
npx nx g @nx/nest:app  
apps/{nome_app}
```

Bootstapping



- File `{projectName}/src/main.ts`

- Contiene il codice «entry-point» per far partire l'intera applicazione di backend (funzione `bootstrap()`)

Moduli «importati»:

- **Logger**: utility NestJS per scrivere log (in questo caso, su standard output)
- **NestFactory**: fabbrica per creare l'app NestJS
- **AppModule**: il modulo root dell'app (dove NestJS inizia a costruire il grafo di moduli/controller/provider)

```
import { Logger } from '@nestjs/common';
import { NestFactory } from '@nestjs/core';
import { AppModule } from '../app/app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  const globalPrefix = 'api';
  app.setGlobalPrefix(globalPrefix);
  const port = process.env.PORT || 3000;
  await app.listen(port);
  Logger.log(
    `🚀 Application is running on: http://localhost:3000`,
  );
}

bootstrap();
```

Bootstapping



- File `{projectName}/src/main.ts`

- Contiene il codice «entry-point» per far partire l'intera applicazione di backend (funzione `bootstrap()`)

Uso di `await/async`:

- `async`: funzione che lavora in modo asincrono (non bloccante) e restituisce una **Promise** (permette di usare `await` al suo interno)
- `await`: sospende l'esecuzione della funzione `async` finché la Promise non si risolve e poi restituisce il valore risolto

```
import { Logger } from '@nestjs/common';
import { NestFactory } from '@nestjs/core';
import { AppModule } from '../app/app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  const globalPrefix = 'api';
  app.setGlobalPrefix(globalPrefix);
  const port = process.env.PORT || 3000;
  await app.listen(port);
  Logger.log(
    `🚀 Application is running on: http://localhost:${port}`,
  );
}

bootstrap();
```


Bootstapping



- File `{projectName}/src/main.ts`

- Contiene il codice «entry-point» per far partire l'intera applicazione di backend (funzione `bootstrap()`)

Imposta un prefisso globale per tutte le rotte:
- d'ora in poi qualsiasi rotta avrà «`/api`» davanti

```
import { Logger } from '@nestjs/common';
import { NestFactory } from '@nestjs/core';
import { AppModule } from '../app/app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  const globalPrefix = 'api';
  app.setGlobalPrefix(globalPrefix);
  const port = process.env.PORT || 3000;
  await app.listen(port);
  Logger.log(
    `🚀 Application is running on: http://localhost:3000`,
  );
}

bootstrap();
```

Bootstapping



- File `{projectName}/src/main.ts`

- Contiene il codice «entry-point» per far partire l'intera applicazione di backend (funzione `bootstrap()`)

Se esiste una variabile d'ambiente `PORT` usa quella, altrimenti imposta la porta su 3000

```
import { Logger } from '@nestjs/common';
import { NestFactory } from '@nestjs/core';
import { AppModule } from '../app/app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  const globalPrefix = 'api';
  app.setGlobalPrefix(globalPrefix);
  const port = process.env.PORT || 3000;
  await app.listen(port);
  Logger.log(
    `🚀 Application is running on: http://localhost:3000`,
  );
}

bootstrap();
```

Uso delle variabili d'ambiente



- Esempio – Nel file `{projectName}/src/main.ts`

- Serve installare preliminarmente il plugin `dotenv`
- Creare il file `.env` nella directory principale del workspace
- Modificare il file `main.ts` importando `dotenv/config`

```
npx nx add dotenv
```

```
PORT=3333
```

```
import 'dotenv/config';
```

Backend app – Il modulo principale



- File `{projectName}/src/app/app.modules.ts`

- È il modulo «root» dell'app di backend
- Da qui **NestJS** inizia a costruire il grafo delle dipendenze e fa partire tutto

```
import { Module } from '@nestjs/common';
import { AppController } from './app.controller';
import { AppService } from './app.service';

@Module({
  imports: [],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}
```

Backend app – Il controller



- File `{projectName}/src/app/app.controller.ts`

- Controller dell'applicazione

Meccanismo di «injection»: il controller verrà utilizzato per invocare i metodi di un servizio (AppService)

```
import { Controller, Get } from '@nestjs/common';
import { AppService } from './app.service';

@Controller()
export class AppController {
  constructor(private readonly appService: AppService) {}

  @Get()
  getData() {
    return this.appService.getData();
  }
}
```

Backend app – Il controller



- File `{projectName}/src/app/app.controller.ts`

- Controller dell'applicazione

Viene esposta una rotta di tipo GET
direttamente all'indirizzo
`http://<host>:<port>/api`

```
import { Controller, Get } from '@nestjs/common';
import { AppService } from './app.service';

@Controller()
export class AppController {
  constructor(private readonly appService: AppService) {}

  @Get()
  getData() {
    return this.appService.getData();
  }
}
```

Backend app – Il controller



- File `{projectName}/src/app/app.controller.spec.ts`

- Controller dell'applicazione
- Viene generato anche il corrispondente `.spec.ts` file per il testing

La libreria `@nestjs/testing` fornisce gli strumenti per creare un modulo NestJS finto per effettuare i test

```
import { Test, TestingModule } from '@nestjs/testing';
import { AppController } from './app.controller';
import { AppService } from './app.service';

describe('AppController', () => {
  let app: TestingModule;

  beforeEach(async () => {
    app = await Test.createTestingModule({
      controllers: [AppController],
      providers: [AppService],
    }).compile();
  });

  describe('getData', () => {
    it('should return "Hello API"', () => {
      const appController = app.get<AppController>(AppController);
      expect(appController.getData()).toEqual({ message: 'Hello API' });
    });
  });
});
```

Backend app – Il controller



- File `{projectName}/src/app/app.controller.spec.ts`

- Controller dell'applicazione
- Viene generato anche il corrispondente `.spec.ts` file per il testing

Setup del modulo di test una sola volta per tutte (`beforeAll`)
La funzione `compile()` compila il modulo, pronto per essere utilizzato (la funzione è asincrona)

```
import { Test, TestingModule } from '@nestjs/testing';
import { AppController } from './app.controller';
import { AppService } from './app.service';

describe('AppController', () => {
  let app: TestingModule;

  beforeAll(async () => {
    app = await Test.createTestingModule({
      controllers: [AppController],
      providers: [AppService],
    }).compile();
  });

  describe('getData', () => {
    it('should return "Hello API"', () => {
      const appController = app.get<AppController>(AppController);
      expect(appController.getData()).toEqual({ message: 'Hello API' });
    });
  });
});
```


Backend app – Il controller



- File `{projectName}/src/app/app.controller.spec.ts`

- Controller dell'applicazione
- Viene generato anche il corrispondente `.spec.ts` file per il testing

Recupero del controller del modulo `app`

```
import { Test, TestingModule } from '@nestjs/testing';
import { AppController } from './app.controller';
import { AppService } from './app.service';

describe('AppController', () => {
  let app: TestingModule;

  beforeEach(async () => {
    app = await Test.createTestingModule({
      controllers: [AppController],
      providers: [AppService],
    }).compile();
  });

  describe('getData', () => {
    it('should return "Hello API"', () => {
      const appController = app.get<AppController>(AppController);
      expect(appController.getData()).toEqual({ message: 'Hello API' });
    });
  });
});
```

Backend app – Il servizio (provider)



- File `{projectName}/src/app/app.service.ts`

- Elemento «*iniettabile*» che contiene la logica di business dell'applicazione

```
import { Injectable } from '@nestjs/common';

@Injectable()
export class AppService {
  getData(): { message: string } {
    return { message: 'Hello API' };
  }
}
```

Backend app – Il servizio (provider)



- File `{projectName}/src/app/app.service.spec.ts`

- Elemento «*iniettabile*» che contiene la logica di business dell'applicazione
- Viene generato anche il corrispondente `.spec.ts` file per il testing

Setup del modulo di test una sola volta per tutte (`beforeAll`)
Si noti che in questo caso viene specificato solo il provider (il controller «viene saltato»)

```
import { Test } from '@nestjs/testing';
import { AppService } from './app.service';

describe('AppService', () => {
  let service: AppService;

  beforeAll(async () => {
    const app = await Test.createTestingModule({
      providers: [AppService],
    }).compile();

    service = app.get<AppService>(AppService);
  });

  describe('getData', () => {
    it('should return "Hello API"', () => {
      expect(service.getData()).toEqual({ message: 'Hello API' });
    });
  });
});
```

Backend app – Il servizio (provider)



- File `{projectName}/src/app/app.service.spec.ts`

- Elemento «*iniettabile*» che contiene la logica di business dell'applicazione
- Viene generato anche il corrispondente `.spec.ts` file per il testing

Recupero del servizio del modulo `app`

```
import { Test } from '@nestjs/testing';
import { AppService } from './app.service';

describe('AppService', () => {
  let service: AppService;

  beforeAll(async () => {
    const app = await Test.createTestingModule({
      providers: [AppService],
    }).compile();

    service = app.get<AppService>(AppService);
  });

  describe('getData', () => {
    it('should return "Hello API"', () => {
      expect(service.getData()).toEqual({ message: 'Hello API' });
    });
  });
});
```

Aggiungere Swagger



- Per testare le API esposte dal backend, può essere utile installare **Swagger**
- Fornisce un'interfaccia Web con l'elenco delle API create e la possibilità di invocarle

```
npx nx add  
@nestjs/swagger
```

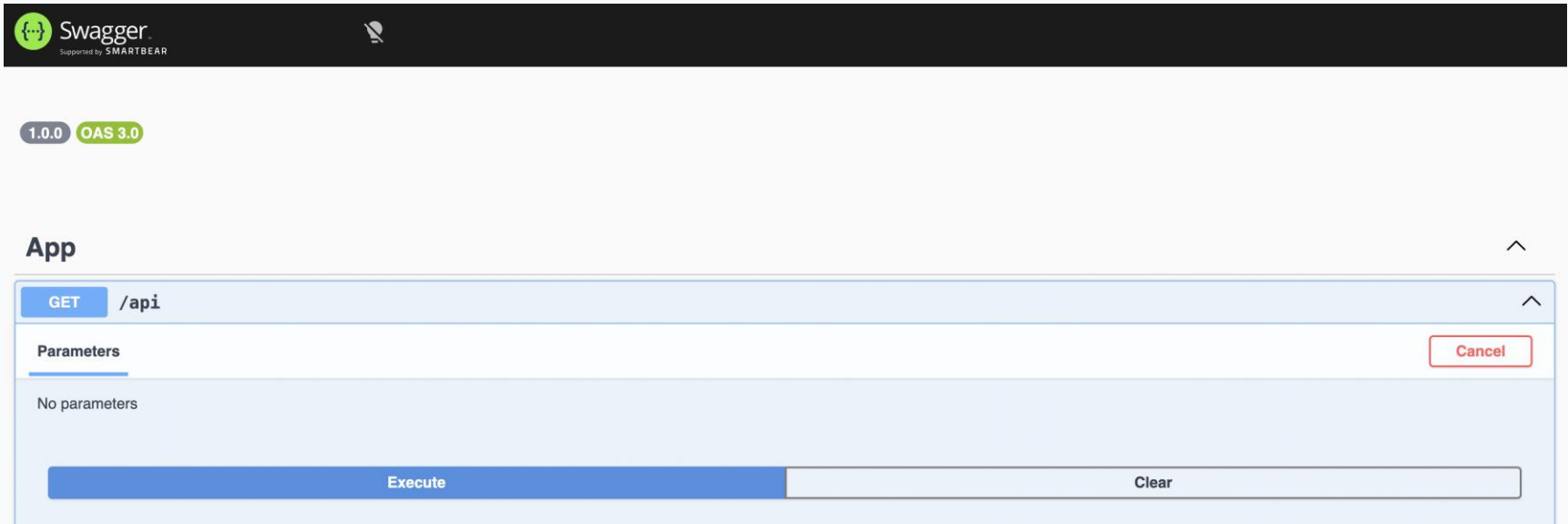
File `{projectName}/src/main.ts` :

```
import { Logger } from '@nestjs/common';  
import { NestFactory } from '@nestjs/core';  
import { AppModule } from '../app/app.module';  
import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger';  
  
async function bootstrap() {  
  const app = await NestFactory.create(AppModule);  
  const globalPrefix = 'api';  
  app.setGlobalPrefix(globalPrefix);  
  
  const config = new DocumentBuilder().build();  
  
  const document = SwaggerModule.createDocument(app, config);  
  
  SwaggerModule.setup('api/docs', app, document);  
  
  const port = process.env.PORT || 3000;  
  await app.listen(port);  
  Logger.log(  
    `🚀 Application is running on: http://localhost:\${port}/\${globalPrefix}`  
  );  
}  
  
bootstrap();
```

Aggiungere Swagger



- L'interfaccia web di **Swagger**: <http://<host>:<port>/api/docs>



Personalizzare Swagger



```
import { ApiTags } from '@nestjs/swagger';

@ApiTags('Example of API')
@Controller()
export class AppController {
  constructor(private readonly appService: AppService) {}

  @Get('data')
  getData() {
    return this.appService.getData();
  }
}
```

Example of API

GET /api/data

Parameters

No parameters

Try it out

Creazione di librerie con NestJS e node



```
npx nx g @nx/nest:library libs/server/users --controller --service
```

- Crea una nuova libreria basta su **NestJS** e **node**, che include anche un *servizio* ed un *controller*
- È anche possibile aggiungere l'opzione **--importPath** per assegnare un nome univoco alla libreria (per esempio, **--importPath=@server/users**)
- Le librerie sono riutilizzabili nelle app (create con il comando visto nella lezione L2 su **Nx**)

```
npx nx g @nx/nest:app apps/{appName}
```

```
npx nx g @nx/nest:library libs/server/users  
npx nx g @nx/nest:controller /libs/server/src/lib/users  
npx nx g @nx/nest:service /libs/server/src/lib/users
```


Node.js: problemi di compatibilità (I)



- **Node.js** (2009) aveva bisogno subito di un modo per «spezzare il codice in moduli» → all'epoca lo standard JS non aveva ancora un sistema in moduli maturo
- Di conseguenza, Node.js adotta inizialmente lo stile **CommonJS (CJS)** → caricamento moduli «a run-time» (utilizzo di **require()**)
- Anni dopo, JavaScript «ufficiale» (**ECMAScript**) introduce lo stile **ES Modules (ESM)** per tutti → struttura **import/export** più dichiarativa, gli import sono noti prima, per ottimizzazione
- **Chi può importare chi?** ESM spesso riesce a caricare CJS, CJS può avere problemi a usare librerie ESM

Node.js: problemi di compatibilità (II)



- L'istruzione **import** è tipica dello stile **ESM**
- Utilizzata in TS
- A livello di sorgente lo stile è **ESM**
- A livello di Javascript (dopo la compilazione), l'uso dello stile **CJS/ESM** viene impostato nel file **package.json** di progetto
 - `'type': 'commonjs'`
 - `'type': 'module'`

```
import { Logger } from '@nestjs/common';  
import { NestFactory } from '@nestjs/core';  
import { AppModule } from './app/app.module';  
import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger';
```

Link e riferimenti utili



- Link alla documentazione NestJS:
<https://docs.nestjs.com/>
- Link alla documentazione Swagger:
<https://swagger.io/doc>
- Dave Gray's tutorials on YouTube



Progettazione di applicazioni web full stack Sviluppo backend con Node e NestJS



Prof. Devis Bianchini
Dip. di Università degli Studi di Brescia